



29. Python: Image Processing

"A picture is worth a thousand words. Pillow helps Python understand them."

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 28. Python: Graphical User Interface with Tkinter](#)

Contents:

[1. Keywords in the Pillow Library](#)

[2. Basic Operations on Pillow](#)

[2.1. Opening and Displaying Images](#)

[2.2. Resizing Images](#)

[2.3. Cropping Images](#)

[2.4. Rotating Images](#)

[2.5. Applying Filters and Enhancements](#)

[2.6. Manipulating Individual Pixels](#)

[2.7. Saving Images](#)

[2.8. Retrieving Image Information and Metadata](#)

[2.9. Working with Colors](#)

[3. Basic Systems Using Pillow](#)

[3.1. Solving an Image Puzzle](#)

[3.2. Image Compression](#)

[3.3. Image Classification](#)

[3.4. A Simple Image Background Remover with Tkinter](#)

[Summary](#)

[References](#)

In the previous section, we explored the Tkinter library and learned how to build graphical user interfaces. We briefly saw that Tkinter can display images, but when it comes to actually editing, analyzing, enhancing, or creating images, we need a more specialized library. That library is **Pillow**.

Pillow is one of the most popular image-processing libraries in Python. It allows us to open image files, resize them, crop them, rotate them, apply filters, draw shapes, add text, manipulate colors, and much more. Whether you are building a photo editor, a machine learning project, a computer vision application, or simply displaying images in a GUI, Pillow is often the first tool developers reach for.

To install Pillow:

```
!pip install pillow
```

Common imports:

```
from PIL import Image
```

Or when working with Tkinter:

```
from PIL import Image, ImageTk
from tkinter import *
```



A Small Note: `import module` vs `from module import *`

One detail that was intentionally left out in the Python Libraries chapter is the difference between: `import tkinter` and `from tkinter import *` (we use `tkinter` as an example. This is applied to literally every module).

- When using: `import tkinter`, every function or class must be prefixed with the module name:

```
window = tkinter.Tk()
button = tkinter.Button(window)
```

- With: `from tkinter import *`, everything is imported directly into the current namespace:

```
window = Tk()  
button = Button(window)
```

This can make GUI code significantly shorter and easier to read because graphical applications often use dozens of widgets and methods repeatedly.

However, for most libraries, using `import module` is generally preferred because it makes the source of each function clearer and helps avoid naming conflicts between different modules.

1. Keywords in the Pillow Library

The Pillow library contains many classes and methods. The table below summarizes some of the most commonly used ones.

Keyword	Description
<code>Image.open()</code>	Opens an image file.
<code>Image.new()</code>	Creates a new blank image.
<code>image.save()</code>	Saves an image to a file.
<code>image.show()</code>	Displays the image using the system viewer.
<code>image.copy()</code>	Creates a duplicate of an image.
<code>image.resize()</code>	Changes the dimensions of an image.
<code>image.thumbnail()</code>	Creates a smaller version while maintaining aspect ratio.
<code>image.crop()</code>	Extracts a rectangular region from an image.
<code>image.rotate()</code>	Rotates an image by a specified angle.
<code>image.transpose()</code>	Flips or rotates an image using predefined operations.
<code>image.convert()</code>	Converts an image between formats or color modes.

Keyword	Description
<code>image.getdata()</code>	Retrieves pixel data from an image.
<code>image.putdata()</code>	Writes pixel data into an image.
<code>image.getpixel()</code>	Retrieves the color of a specific pixel.
<code>image.putpixel()</code>	Changes the color of a specific pixel.
<code>ImageTk.PhotoImage()</code>	Converts a Pillow image into a Tkinter-compatible image.
<code>ImageFilter.BLUR</code>	Applies a blur effect.
<code>ImageFilter.SHARPEN</code>	Sharpens an image.
<code>ImageFilter.CONTOUR</code>	Highlights image contours and edges.
<code>ImageFilter.EMBOSS</code>	Creates an embossed effect.
<code>ImageEnhance.Brightness</code>	Adjusts image brightness.
<code>ImageEnhance.Contrast</code>	Adjusts image contrast.
<code>ImageEnhance.Color</code>	Adjusts color intensity.
<code>ImageDraw.Draw()</code>	Creates a drawing object for adding shapes and text.
<code>draw.line()</code>	Draws a line on an image.
<code>draw.rectangle()</code>	Draws a rectangle.
<code>draw.ellipse()</code>	Draws circles and ellipses.
<code>draw.text()</code>	Writes text onto an image.
<code>ImageFont.truetype()</code>	Loads a custom font for drawing text.

As with many Python libraries, you do not need to memorize every method. The important part is understanding what Pillow can do and knowing where to find the right tool when you need it. Later sections will explore these keywords through practical examples.

2. Basic Operations on Pillow

Now that we know what Pillow is, it's time to actually use it.

Think of Pillow as a toolbox for images. Just like Python can manipulate numbers, strings, and files, Pillow allows Python to manipulate pictures. Whether you want to resize a photo, crop unwanted sections, apply artistic

effects, or even edit individual pixels, Pillow gives you direct access to the image itself.

And the best part? Pillow works exceptionally well with Tkinter, allowing us to build complete image-editing applications with only a few lines of code.

Let's start with the most common operations. Here, I will include some images in case you want to try these Pillow operations below.

cat.jpg

dog.jpg

fish.jpg

(There will be no image outputs for the codes in this section)

2.1. Opening and Displaying Images

Before we can edit an image, we must first load it into memory. There are two common approaches:

- **Method 1: Direct Path**

```
from PIL import Image

img = Image.open("cat.jpg")
img.show()
```

The `show()` method opens the image using your computer's default image viewer.

- **Method 2: Using Tkinter File Dialog**

```

from tkinter import *
from tkinter import filedialog
from PIL import Image

file = filedialog.askopenfilename()

if file:
    img = Image.open(file)
    img.show()

```

This allows users to choose an image dynamically.

- **Displaying Images Inside Tkinter:** Instead of opening a separate viewer, we can display the image directly inside a GUI.

```

from tkinter import *
from PIL import Image, ImageTk

root = Tk()

img = Image.open("cat.jpg")
photo = ImageTk.PhotoImage(img)

label = Label(root, image=photo)
label.pack()

root.mainloop()

```

The `ImageTk.PhotoImage()` class converts a Pillow image into a format Tkinter can understand.

2.2. Resizing Images

Sometimes an image is simply too large. Pillow provides two main methods:

Method	Description
<code>resize()</code>	Resizes to exact dimensions
<code>thumbnail()</code>	Resizes while preserving aspect ratio

- Example 1: The image is forced into the specified dimensions.

```
from PIL import Image

img = Image.open("cat.jpg")

new_img = img.resize((300, 200))
new_img.show()
```

- Example 2: Unlike `resize()`, the image keeps its original proportions. Think of `thumbnail()` as creating a miniature version of the image.

```
from PIL import Image

img = Image.open("cat.jpg")

img.thumbnail((300, 300))
img.show()
```

2.3. Cropping Images

Cropping allows us to extract only the region we need.

```
from PIL import Image

img = Image.open("cat.jpg")

cropped = img.crop((100, 50, 400, 300))
cropped.show()
```

The four values represent: `(left, top, right, bottom)`. Everything inside the rectangle is preserved. Everything outside is discarded.

2.4. Rotating Images

Images can be rotated by any angle.

```
from PIL import Image

img = Image.open("cat.jpg")

rotated = img.rotate(90)
rotated.show()
```

0° → Original

90° → Quarter Turn

180° → Upside Down

270° → Three-Quarter Turn

You may also rotate using arbitrary angles: `img.rotate(45)`

2.5. Applying Filters and Enhancements

One of Pillow's most powerful features is image enhancement.

- **Blurriness**

```
from PIL import Image, ImageFilter

img = Image.open("cat.jpg")

blurred = img.filter(ImageFilter.BLUR)
blurred.show()
```

- **Sharpen**


```
sharpened = img.filter(ImageFilter.SHARPEN)
sharpened.show()
```

- **Brightness**

```
from PIL import ImageEnhance

img = Image.open("cat.jpg")

enhancer = ImageEnhance.Brightness(img)

bright = enhancer.enhance(1.5)
bright.show()
```

1.0 = Original

>1 = Brighter

<1 = Darker

- **Color Enhancement**

```
from PIL import ImageEnhance

enhancer = ImageEnhance.Color(img)

colorful = enhancer.enhance(2)
colorful.show()
```

This increases color intensity.

- **Contrast**

```
enhancer = ImageEnhance.Contrast(img)
```

```
contrast = enhancer.enhance(2)
contrast.show()
```

Useful for making details stand out.

2.6. Manipulating Individual Pixels

At the deepest level, images are simply collections of pixels. Pillow allows us to access and modify them directly.

- **Reading a Pixel**

```
from PIL import Image

img = Image.open("cat.jpg")

pixel = img.getpixel((100, 100))
print(pixel)
```

(172, 175, 182)

This represents RGB pixel → (red, blue, green). The maximum number for each value is 255.

- **Changing a Pixel**

```
img.putpixel((100,100), (0,255,0))
img.show()
```

The selected pixel becomes green.

- **Drawing a Red Line**

```
for x in range(100):
    img.putpixel((x,50), (255,0,0))
```

```
img.show()
```

This manually colors pixels one by one. Not very efficient—but useful for understanding how digital images work internally.

2.7. Saving Images

After editing, we usually want to save the result. Pillow automatically determines the format from the extension.

- Save as JPEG:

```
img.save("photo.jpg")
```

- Save as PNG:

```
img.save("photo.png")
```

- Save as BMP:

```
img.save("photo.bmp")
```

The same image can be converted into many different formats.

2.8. Retrieving Image Information and Metadata

Pillow can also tell us information about an image.

```
from PIL import Image

img = Image.open("cat.jpg")

print(img.size)
print(img.mode)
print(img.format)
```

(960, 1282)

RGB

JPEG

We can also retrieve raw pixel data.

```
pixels = list(img.get_flattened_data())
print(pixels[:10])
```

[(155, 161, 173), (155, 161, 173), (153, 161, 174), (154, 162, 175), (155, 164, 179), (158, 167, 182),
(159, 171, 187), (161, 173, 189), (161, 172, 190), (162, 173, 191)]

The `get_flattened_data()` method returns all pixels in the image. This is commonly used in Computer Vision; Machine Learning; Image Analysis; Pattern Recognition, because it gives direct access to the underlying numerical representation of an image.

2.9. Working with Colors

One of the most interesting aspects of image processing is color manipulation.

Remember that every color image is simply a collection of pixels, and every pixel stores numerical color information. By modifying these values, we can completely change the appearance of an image. Let's explore some of the popular color manipulation techniques.

- Converting to Grayscale: Grayscale images contain only brightness information.

```
from PIL import Image

img = Image.open("cat.jpg")

gray = img.convert("L")
gray.show()
```

Black ← Shades of Gray → White

This is commonly used in: Computer Vision; Edge Detection; OCR (Text Recognition); Machine Learning because it reduces the amount of information that must be processed.

- **Converting to HSV:** RGB is designed for displays. HSV is designed more around how humans perceive colors.

```
from PIL import Image

img = Image.open("cat.jpg")

hsv = img.convert("HSV")
hsv.show()
```

HSV stands for: H → Hue (actual color); S → Saturation (color intensity); V → Value (brightness). HSV is often easier when working with color detection and segmentation.

- Creating a Simple Mask

A mask allows us to separate bright and dark regions.

```

from PIL import Image

img = Image.open("cat.jpg")

gray = img.convert("L")

mask = gray.point(lambda x: 255 if x > 120 else 0)
mask.show()

```

White = Selected

Black = Ignored

Masks are frequently used in: Object Detection; Background Removal; Image Segmentation

- **Extracting a Specific Color Channel**

Since RGB images contain three channels: R = Red; G = Green; B = Blue. We can isolate individual channels.

- Red Version

```

from PIL import Image

img = Image.open("cat.jpg")

r, g, b = img.split()
blank = g.point(lambda x: 0)

red_img = Image.merge("RGB", (r, blank, blank))
red_img.show()

```

Only red components remain visible.

- Green Version

```

green_img = Image.merge("RGB", (blank, g, blank))
green_img.show()

```

- Blue Version

```
blue_img = Image.merge("RGB", (blank, blank, b))
blue_img.show()
```

This is an excellent way to understand how RGB images are constructed internally.

- **Recoloring an Entire Image**

Suppose we want every pixel to become blue-tinted.

```
from PIL import Image

img = Image.open("cat.jpg")

r, g, b = img.split()

blue_img = Image.merge("RGB", (r.point(lambda x: x // 2),
                                g.point(lambda x: x // 2),
                                b.point(lambda x: min(x + 10,
0, 255)))))

blue_img.show()
```

Original image + Strong blue tint

This technique is commonly used in: Filters; Artistic Effects; Theme Generation; Photo Editing Apps

- **Advanced: Changing the Color of a Specific Object**

Now things become much more interesting. Instead of changing the entire image, we may want to change only one object. For example: *Orange Cat* → *Blue Cat*

Humans immediately recognize the cat. Computers do not. The computer must first identify which pixels belong to the cat. This process is called

Segmentation or **Object Detection**

Suppose our cat is mostly orange. Orange pixels generally have: High Red; Medium Green; Low Blue

We can search for pixels matching this pattern.

```
from PIL import Image

img = Image.open("cat.jpg")

pixels = img.load()

width, height = img.size

for x in range(width):
    for y in range(height):
        r, g, b = pixels[x, y]
        if r > 150 and g > 80 and b < 100:
            pixels[x, y] = (0, 0, 255)

img.show()
```

Orange regions become blue, but not all. To do such thing, a clustering model might be required or another more powerful library like OpenCV

This simple recoloring example introduces a surprisingly powerful idea: We are no longer editing the image. We are editing selected pixels based on their properties. This concept forms the foundation of: Green Screen Effects; Snapchat Filters; Face Tracking; Background Removal; Autonomous Vehicle Vision; Medical Image Analysis

In professional computer vision systems, AI models perform the segmentation automatically. But underneath all those advanced systems lies the same idea: ***Find the pixels*** → ***Modify the pixels*** → ***Create a new image***. And once you understand that, you begin to see images not as pictures—but as millions of tiny pieces of data waiting to be manipulated.

At this point, we can already perform most common image-editing tasks. And this is only scratching the surface. In the next sections, we'll combine Pillow with Tkinter to build actual image-processing applications that users can interact with visually.

3. Basic Systems Using Pillow

Learning individual Pillow functions is useful. But just like Tkinter, the real understanding comes when we combine multiple concepts into a small project.

And one of the best beginner projects is surprisingly simple: ***Can a computer tell whether two images are exactly the same?***

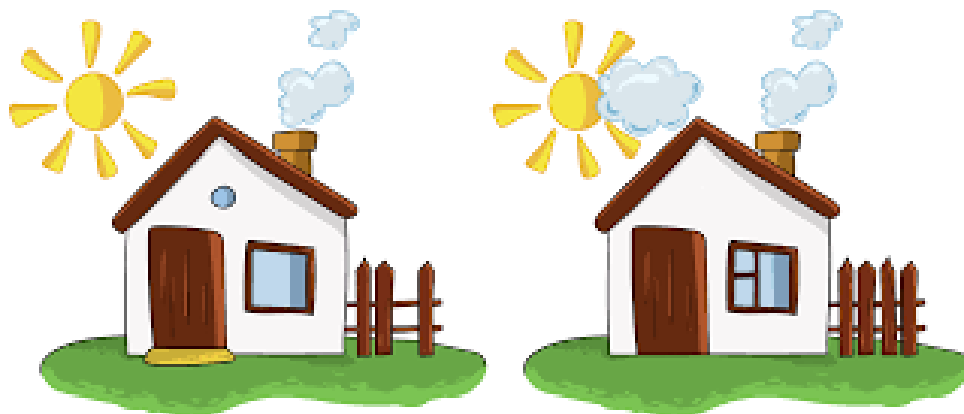
At first, this sounds trivial. You look at two pictures and immediately know whether they match. But computers don't see images the way humans do.

- Humans see houses, trees, cars, and faces
- Computers see numbers, pixels, arrays

To a computer, an image is simply a gigantic table of pixel values. And that leads us to our first system.

3.1. Solving an Image Puzzle

Imagine someone gives you two pieces of a puzzle: `house_left.jpg` and `house_right.jpg`



Source: <https://realpython.com/image-processing-with-the-python-pillow-library/>

At first glance, they may look identical. But are they really? Let's ask Python.

```
from PIL import Image
import numpy as np

with Image.open("house_left.jpg") as left:
    left.load()

with Image.open("house_right.jpg") as right:
    right.load()

left_array = np.asarray(left)
right_array = np.asarray(right)

print(type(left_array))
print(type(right_array))
```

```
<class 'numpy.ndarray'>
```

```
<class 'numpy.ndarray'>
```

Notice something important. The images are no longer images. They have become NumPy arrays. This means every pixel is now represented as numerical data that Python can analyze.

For example:

```
print(left_array.shape)
```

```
(800, 1200, 3) → Meaning: 800 rows, 1200 columns, 3 color channels (RGB)
```

The image is now just a giant matrix.

- **Comparing Two Images:** Now we can compare them directly.

```
same = np.array_equal(left_array, right_array)
print(same)
```

True or False

- If the result is `True`, every single pixel matches. The images are completely identical.
- If the result is `False`, at least one pixel is different. And that's enough for the computer to consider them different images. Even if humans cannot immediately spot the difference.

Finding the Difference

We can even count how many pixels differ.

```
difference = np.sum(left_array != right_array)
print("Different values:", difference)
```

Sample Output:

Different values: 12

This means only 12 pixel values are different. To us? The images may look exactly the same. To the computer? They are two separate images.

Why This Is a Great Beginner Problem

This tiny project teaches one of the most important concepts in image processing: *Images are data*. When beginners first work with images, they often think in terms of objects: This is a house. This is a tree. This is a cat.

But computers don't understand any of those things. Instead, they see: **(255, 0, 0); (100, 150, 200); (34, 75, 210)**

Millions of tiny RGB values arranged in a grid. By converting images into NumPy arrays and comparing pixels, you begin thinking the way image-processing systems think. This idea forms the foundation of: Computer Vision; Facial Recognition; Medical Imaging; Image Compression; Machine Learning; Artificial Intelligence

In fact, many advanced AI systems start with exactly the same idea:

Take an image → Convert it into numbers → Analyze the numbers → Discover patterns.

So while this puzzle appears simple, it teaches a very powerful lesson: ***To humans, an image is a picture. To a computer, an image is mathematics.***

3.2. Image Compression

As cameras become more powerful, image sizes continue to grow.

A modern smartphone photo can easily be several megabytes in size. While high-quality images are useful, they also consume more storage space, memory, network bandwidth, loading time

Image compression attempts to reduce file size while preserving as much visual quality as possible. This is one of the most common tasks in image processing.

- A Simple Image Compression Program

```
from PIL import Image

def compress_image(input_path, output_path, target_quality=85):

    with Image.open(input_path) as img:
        if img.mode in ("RGBA", "P"):
            img = img.convert("RGB")

        img.save(output_path, "JPEG",
                 quality=target_quality, optimize=True)

compress_image("cat.jpg", "compressed_catt.jpg", target_quality=75)
```

- First, we load the image using `with Image.open(input_path) as img:`. If the image contains transparency: ***RGBA***, JPEG cannot store the alpha channel.

Therefore we convert it with `img.convert("RGB")`. Finally, we save the image with reduced quality.

- The Quality Parameter: `quality=75` controls how aggressively Pillow compresses the image. For most applications, around 75 to 85 provides a good compromise between image quality and file size.

Quality	Result
100	Maximum quality, larger file
90	Excellent quality
75	Good balance
50	Noticeable compression
20	Heavy quality loss

Interestingly, many images contain information that humans barely notice. For example: Tiny color variations; Extremely fine details; Small texture changes.

Compression algorithms remove some of this information. Humans still see a cat, while the computer stores significantly less data. This is why platforms such as: Facebook; Instagram; Discord; WhatsApp automatically compress uploaded images.

3.3. Image Classification

One of the most fascinating applications of image processing is teaching a computer to recognize what appears inside an image.

For example: Cat; Dog; Car; Airplane; Banana

Humans do this effortlessly. Computers require a model that has learned patterns from thousands or even millions of examples.

This field belongs to Machine Learning and Artificial Intelligence. However, since this is a Python course, we will focus only on using the model rather than learning how the model itself is trained.

- **A Pretrained Image Classifier:** The following example uses MobileNetV2, a lightweight image classification model.

```

import numpy as np
from PIL import Image
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

model = MobileNetV2(weights='imagenet')
image_path = "cat.jpg"

img = Image.open(image_path)
img = img.resize((224, 224))

img_array = np.array(img)
img_array = np.expand_dims(img_array, axis=0)

img_array = preprocess_input(img_array)
predictions = model.predict(img_array)

decoded_predictions = decode_predictions(predictions, top=3)[0]

print(f"Predictions for {image_path}:")

for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}. {label.capitalize()} ({score * 100:.2f}%)")

```

Predictions for cat.jpg:

1. Tiger_cat (45.70%)
2. Tabby (18.50%)
3. Egyptian_cat (8.95%)

The model is not simply saying it is a tiger cat. Instead, it estimates probabilities for thousands of possible categories and returns the most likely answers.

- **Why Resize to 224 × 224?** MobileNetV2 expects images of a fixed size. So `img.resize((224, 224))` ensures every image has the same dimensions before being analyzed. Think of it as giving every student the same exam paper size before grading.
- **Why Convert to NumPy?** The model does not understand Pillow images directly. It expects numerical tensors. Therefore, `np.array(img)` converts the image into numbers. Remember our earlier discussion: *To a human, an image is a picture. To a computer, an image is mathematics.* The classifier can only work with the mathematical representation.

Although this example uses only a few lines of code, the same principle powers: Facial Recognition; Medical Diagnosis Systems; Self-Driving Cars; Security Cameras; Wildlife Monitoring; Product Recognition

Modern AI systems often begin with: **Image** → **Pixels** → **Numbers** → **Prediction**

And Pillow serves as the bridge between the image and the numerical data used by machine learning models.

3.4. A Simple Image Background Remover with Tkinter

Now let's build something that feels like a real application. One of the most common image-editing tasks today is removing the background from an image.

Applications like Canva, Photoshop, and many AI-powered tools can automatically separate the foreground object from its background.

Building a professional background remover requires Machine Learning and Computer Vision, which we will learn much later.

But for now, we can create a simplified version using Pillow and Tkinter.

The idea is simple:

1. Select an image using a file dialog.
2. Find pixels that are close to white.
3. Make those pixels transparent.
4. Save the result as a PNG.

This works well for images that already have a mostly white background.

The Complete Program

```
# Background Remover System
from tkinter import *
from tkinter import filedialog, messagebox
from PIL import Image, ImageTk
from rembg import remove # pip install "rembg[cpu]"

selected_file = None
original_image = None

def open_image():
    global selected_file, original_image

    selected_file = filedialog.askopenfilename(filetypes=
[("Images", "*.png *.jpg "
"*.jpeg *.bmp *.webp")])

    if not selected_file:
        return

    original_image = Image.open(selected_file)
    show_preview()

def show_preview():
    preview_window = Toplevel(root)
    preview_window.title("Preview Image")

    img = original_image.copy()

    # Resize image to fit preview window
    max_width = 500
    max_height = 400

    img.thumbnail((max_width, max_height))
```



```

photo = ImageTk.PhotoImage(img)

image_label = Label(preview_window, image=photo)
image_label.image = photo
image_label.pack(padx=10, pady=10)

Button(preview_window, text="Remove Background", font=
("Arial", 12),
        command=lambda: remove_background(preview_windo
w)).pack(pady=10)

def remove_background(window):
    global selected_file

    try:
        with open(selected_file, "rb") as input_file:
            input_data = input_file.read()

            output_data = remove(input_data)

            save_path = filedialog.asksaveasfilename(defaulttext
ension=".png",
                                                    filetypes=
[("PNG Image", "*.png")])

            if save_path:
                with open(save_path, "wb") as output_file:
                    output_file.write(output_data)

                messagebox.showinfo("Success",
                                    "Background removed success
fully!")

                window.destroy()

    except Exception as e:

```

```
messagebox.showerror("Error", str(e))
```

```
# Main Window
```

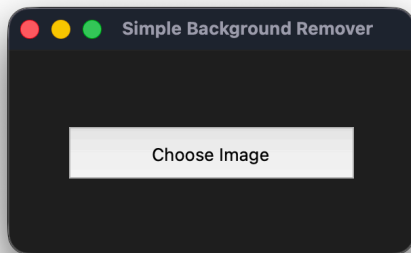
```
root = Tk()
```

```
root.title("Simple Background Remover")
```

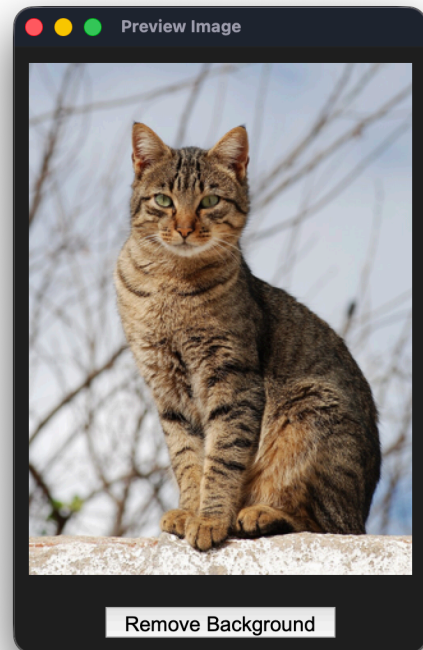
```
root.geometry("300x150")
```

```
Button(root, text="Choose Image", width=20, height=2, command=open_image).pack(expand=True)
```

```
root.mainloop()
```



The main window



The pop-up window to preview the image

Output:

cat_removed_bg.png

How the Program Works

- Import Tkinter for the GUI, Pillow for image handling, and `rembg` for AI background removal.
- The user clicks **Choose Image** to select an image file.
- `filedialog.askopenfilename()` opens a file browser.
- The selected image is loaded using `Image.open()`.
- A new `Toplevel()` window appears to preview the image.
- The image is resized using `thumbnail()` so it fits inside the preview window.
- `ImageTk.PhotoImage()` converts the Pillow image into a Tkinter-compatible image.
- The image is displayed inside a `Label`.
- The user clicks **Remove Background**.
- The image file is read as binary data.
- `remove()` uses an AI segmentation model to separate the foreground from the background.
- The user chooses where to save the result.
- The processed image is saved as a transparent PNG.
- A success message is displayed.
- The preview window closes.
- If an error occurs, an error message is shown instead.

Summary

The Pillow library is one of Python's most powerful tools for image processing.

Throughout this chapter, we learned how to:

- Open and display images
- Resize and crop images
- Rotate images

- Apply filters and enhancements
- Manipulate individual pixels
- Save images in different formats
- Extract image information and metadata
- Work with color channels and masks
- Recolor images and objects
- Compare images using NumPy arrays
- Compress images to reduce file size
- Prepare images for machine learning systems
- Create a simple background remover that integrates with Tkinter GUI system

Perhaps the most important lesson is that images are not magical objects. Behind every photograph lies a massive collection of pixels, and behind every pixel lies numerical data.

Pillow gives us direct access to that data. Whether we are building a simple photo editor, a computer vision application, or an artificial intelligence system, many workflows begin with the same fundamental process:

Open Image → Manipulate Pixels → Analyze Data → Produce Results

And once you understand this pipeline, you begin to see images the same way computers do—not merely as pictures, but as structured information that can be transformed, analyzed, and understood through Python.

References

<https://python-pillow.github.io/>

<https://realpython.com/image-processing-with-the-python-pillow-library/>

<https://www.geeksforgeeks.org/python/python-pillow-tutorial/>

<https://www.codecademy.com/article/getting-started-with-image-processing-in-python-using-pillow>

Next Section:

[`</>` 30.1. Python: NumPy](#)